ICMC USP
SÃO CARLOS

Introdução ao desenvolvimento de aplicativos móveis

Profa. Dra. Lina Garcés

Conteúdo

01 O que é desenvolvimento de Apps móveis?

02 Ecossistema de desenvolvimento de Apps móveis

03 Introdução ao React Native

04 Arquitetura do React Native

05 Projeto em React Native

01

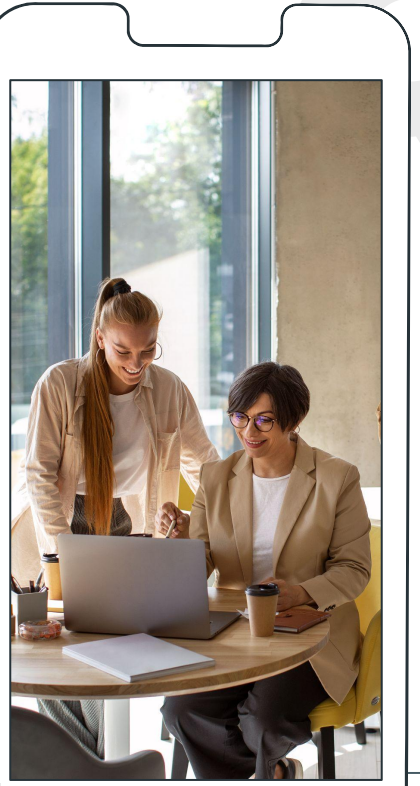
Desenvolvimento de Apps móveis

O que é desenvolvimento de aplicativos móveis?

O desenvolvimento de aplicativos móveis é o processo de criação de software para smartphones, tablets e assistentes digitais, geralmente para os sistemas operacionais Android e iOS.

O software pode ser pré-instalado no dispositivo, baixado de uma loja de aplicativos móveis ou acessado por meio de um navegador da Web de dispositivo móvel.

As linguagens programação e markup utilizadas para esse tipo de desenvolvimento de software podem incluir Java, JavaScript, Swift, C#, Dart e HTML5, entre outras.



https://www.ibm.com/br-pt/topics/mobile-application-development

Frameworks de Desenvolvimento

- Mobile
  - Multiplataforma* — Flutter, React Native, Kotlin Multiplatform Ionic, NativeScript, Xamarin, Cordova
  - iOS — SwiftUI, UIKit
  - Android — Android Studio
- Web — React, Angular, Vue.js, Svelte, JQuery, Semantic UI, Ember, Foundation, Preact, ...
- Back-end — ASP.NET Core, Django, Laravel, Rails, Express.js, Flask, Spring Boot, Phoenix ...

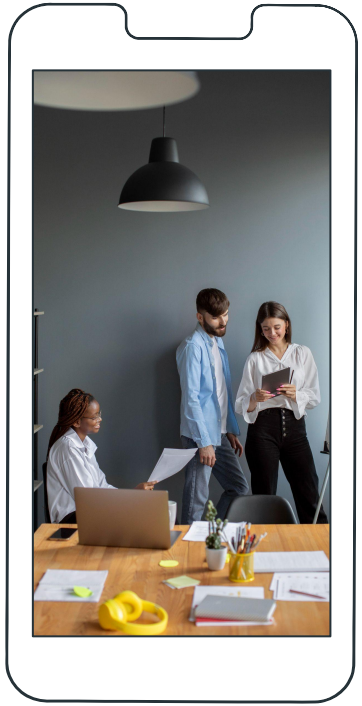
miro

* Alguns frameworks também são usados para desenvolvimento de Apps híbridos e/ou web progressivos

Tipos de Apps móveis

Apps Nativos

Apps Híbridos



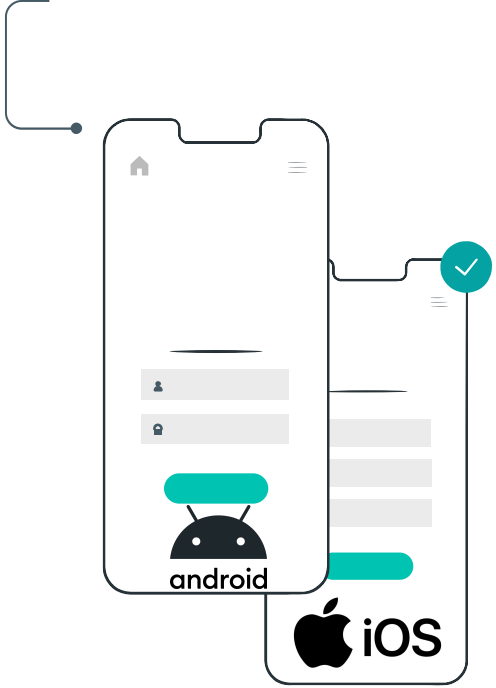
Apps Web progressivos

Apps Multiplataforma

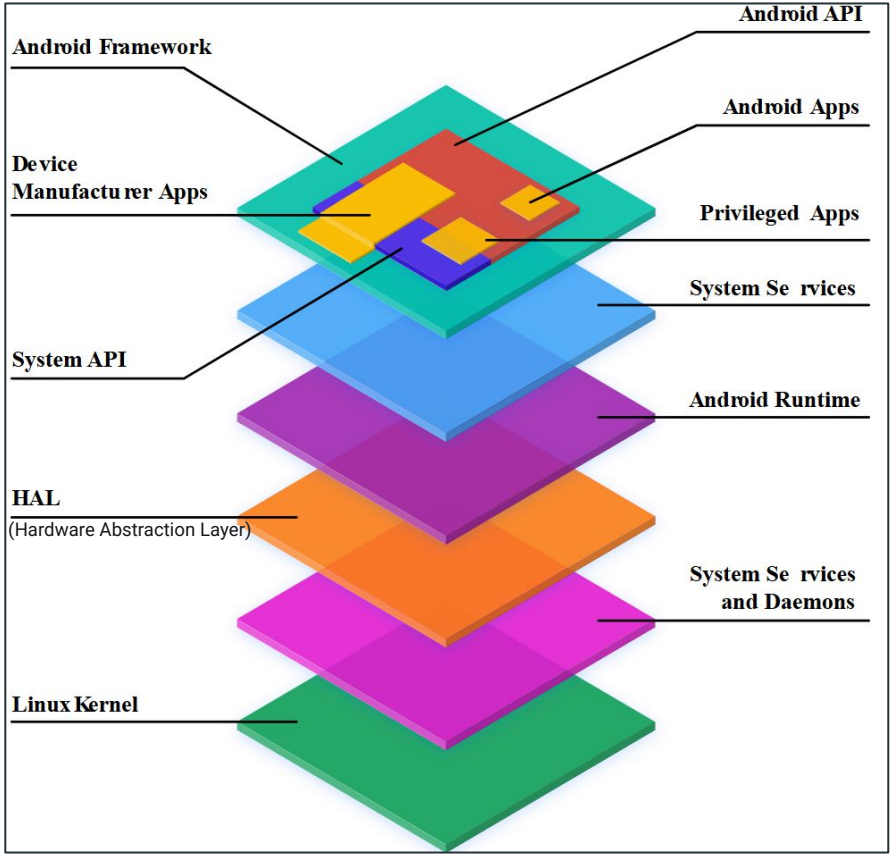
Aplicativos nativos

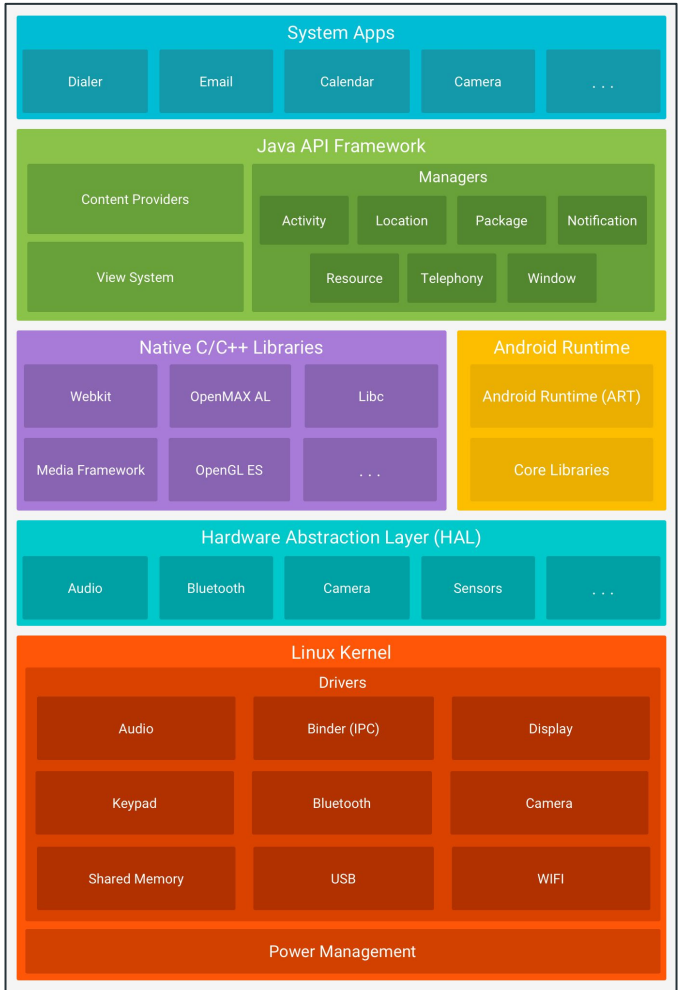
Desenvolver dois aplicativos nativos (Android e iOS): Aproveitar as APIs nativas e linguagens de programação específicas ao SO podem ajudá-lo a construir um aplicativo poderoso. A maioria dos aplicativos corporativos, especialmente aqueles que requerem tráfego considerável de APIs, se beneficiam de desenvolvimento nativo.

Recomenda-se iniciar por Android para desenvolver aplicativos completos como MVP no Android e, em seguida, converter (recodificar) e otimizá-lo para iOS depois do lançamento. *Reúso de back-end na outra plataforma.*



Arquitetura do AOSP (Android Open Source Project)





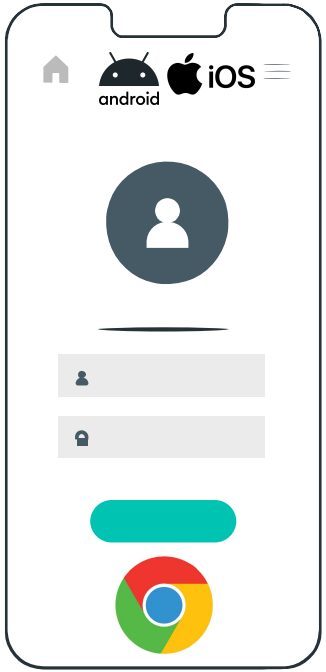
Explicação completa: [Visão geral da arquitetura | Android Open Source Project](https://developer.android.com/guide/platform?hl=pt-br)
https://developer.android.com/guide/platform?hl=pt-br

Aplicativos Web progressivos

Assim como os aplicativos Web, os aplicativos Web progressivos **são executados em navegadores móveis** e normalmente são criados usando JavaScript, CSS e HTML5.

O que torna os aplicativos Web progressivos é a capacidade de fornecer notificações por push, melhores gestos de toque e interação de hardware usando recursos avançados do navegador.

Não há **SDKs*** para ajudar a criar aplicativos Web progressivos, mas eles podem ser implantados sem passar por um marketplace.



* O kit de desenvolvimento de software (SDK) é uma coleção de ferramentas de desenvolvimento de software em um pacote instalável. Eles facilitam a criação de aplicações por terem um compilador, um depurador e às vezes uma estrutura de software.

Aplicativos Híbridos

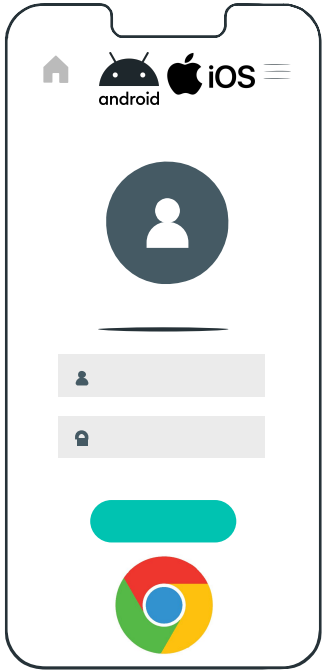
Uma **combinação entre aplicativos nativos e aplicativos Web progressivos**.

Os aplicativos híbridos são essencialmente **contêineres ou webviews codificados nativamente que executam aplicativos Web.**

O código é **desenvolvido como uma aplicação web, mas é distribuído como aplicativo nativo**, usando uma camada adicional para acessar funcionalidades específicas do dispositivo, como câmera e GPS.

Essa combinação permite que o aplicativo tenha mais acesso aos recursos e hardware dos dispositivos e seja executado em mais tipos de dispositivos, mantendo uma única base de código.

Apesar de todas as suas vantagens, os **aplicativos híbridos normalmente não alcançam o desempenho que os aplicativos nativos ou multiplataforma conseguem.**



APIs Nativas

API - application programming interface

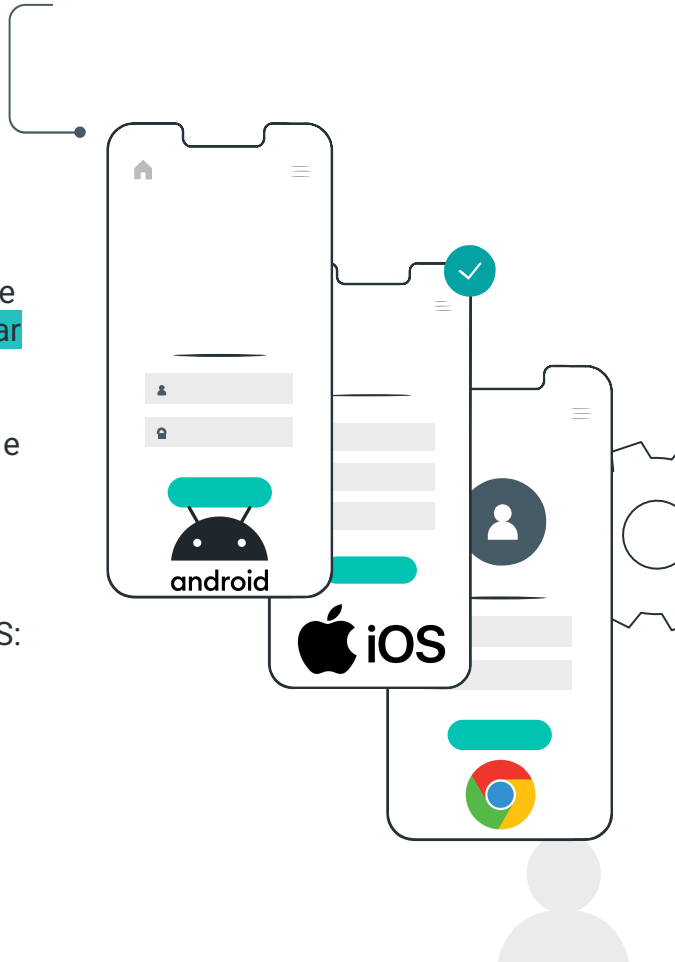
API é um software intermediário que permite que dois aplicativos se comuniquem. APIs são uma forma acessível de extrair e compartilhar dados entre as aplicações.

APIs nativas são fornecidas por um fornecedor de software de plataforma e integradas a um produto.

APIs nativas Android: https://developer.android.com/ndk/reference

APIs nativas iOS: https://developer.apple.com/documentation/technologies

APIs nativas Web: https://developer.mozilla.org/en-US/docs/Web/API



Aplicativos Híbridos com Cordova/Ionic

Cordova Application

Web App

HTML

config.xml

JS

CSS

Resources

Cordova Plugins

Accelerometer

Geolocation

Camera

Media

Device

Network

Contacts

Storage

Custom Plugins

HTML APIs

Cordova APIs

HTML Rendering engine(WebView)

Cordova Native APIs

OS APIs

OS APIs

Mobile OS

Services

Input

Sensors

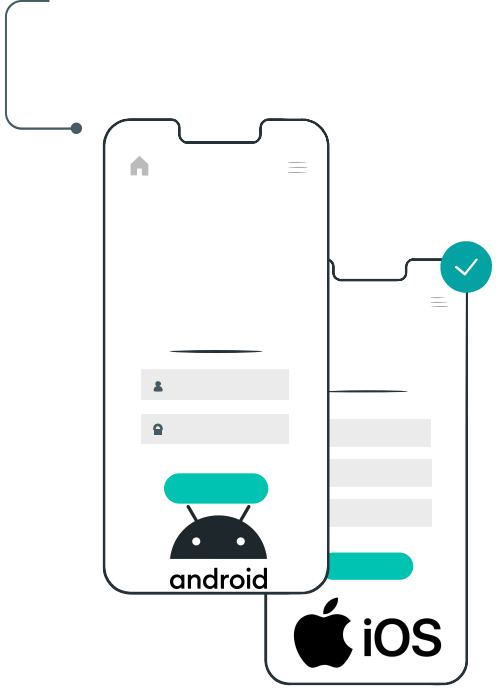
Graphics

Aplicativos Multiplataforma

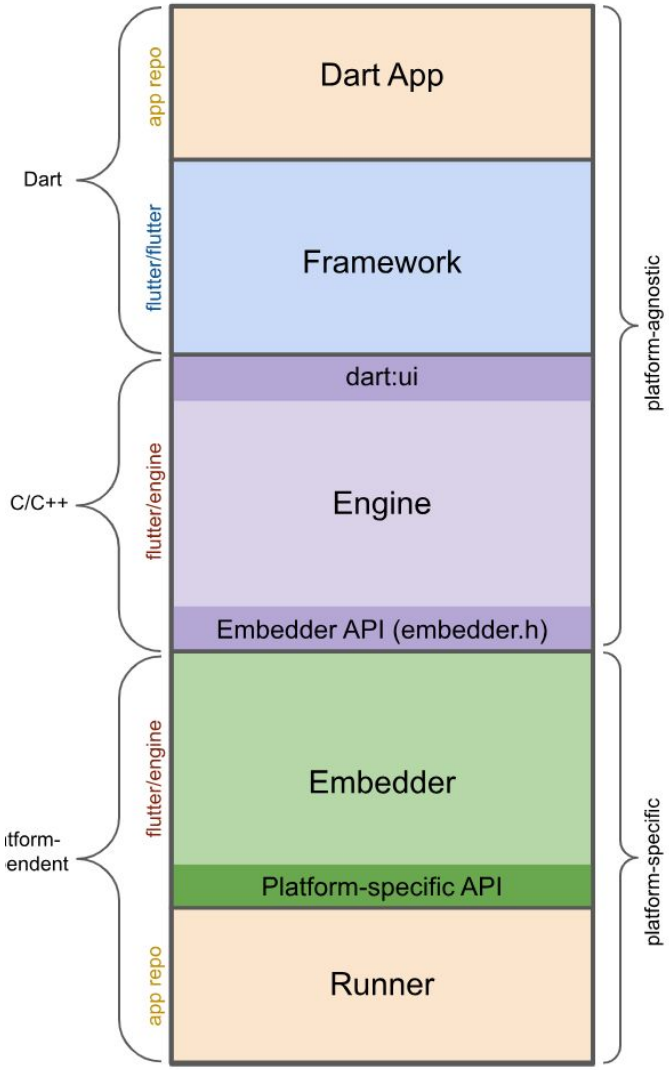
Desenvolvimento Multiplataforma: Abordagem de **criação única e execução em qualquer lugar.**

Os aplicativos multiplataforma usam um único código base que pode funcionar em qualquer plataforma. Eles são normalmente desenvolvidos em uma linguagem de programação universalmente reconhecida, como Java, JavaScript, HTML ou CSS.

Como **o acesso às APIs nativas do sistema operacional não é liberado,** o desenvolvimento multiplataforma de aplicativos móveis funciona melhor para aplicativos da Web simples, como aplicativos móveis de três ou quatro páginas com funcionalidade limitada.



Arquitetura de um App Multiplataforma em Flutter



Explicação completa: https://docs.flutter.dev/resources/architectural-overview

React Native e o conceito de View

No desenvolvimento Android e iOS, uma **view** é o bloco de construção básico da UI: um pequeno elemento retangular na tela que pode ser usado para exibir texto, imagens ou responder à entrada do usuário.

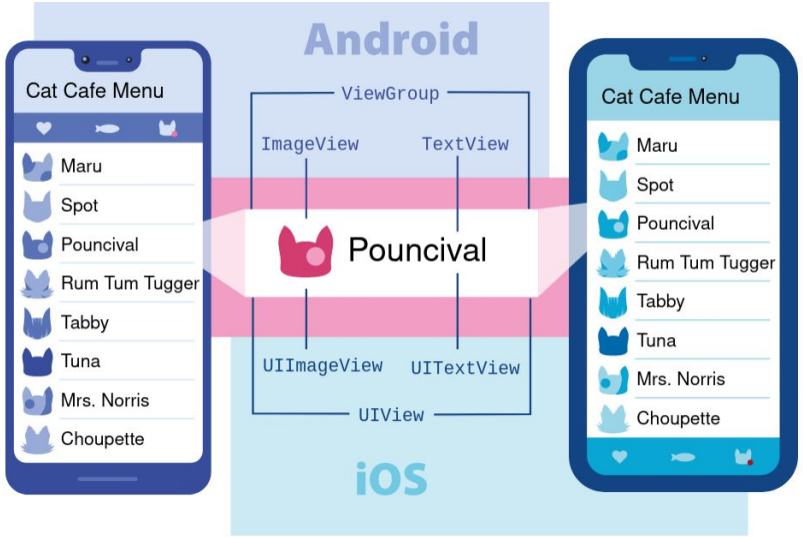
Mesmo os menores elementos visuais de um aplicativo, como uma linha de texto ou um botão, são tipos de **views**.

Alguns tipos de **views** podem conter outras **views**. São **views** até o fim!

Para desenvolvimento em Android as **views** são programadas em Kotlin ou Java.

Em iOS é usado Swift ou Objective-C.

Em RN as **views** são programadas com JavaScript usando **componentes** do framework React.



Just a sampling of the many views used in Android and iOS apps.

https://reactnative.dev/docs/intro-react-native-components

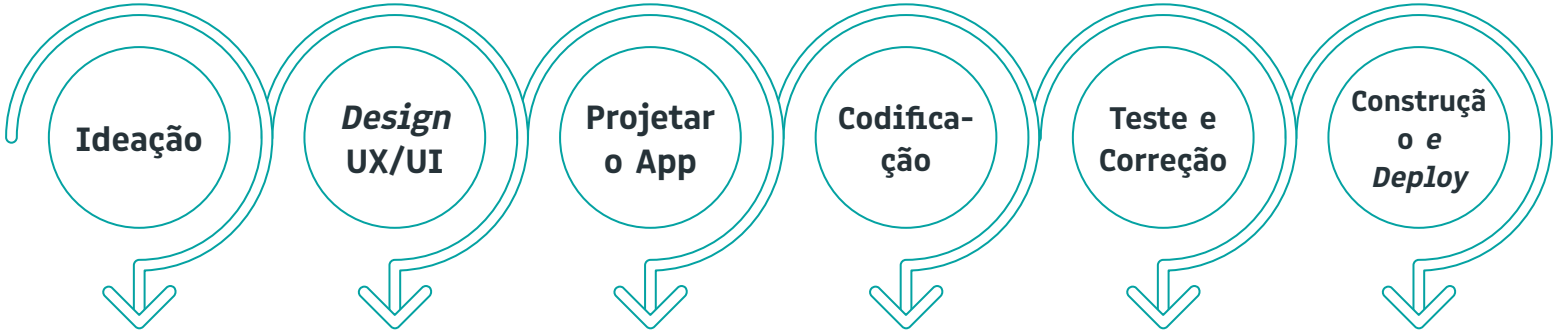
Componentes Core em RN

Componente de UI do RN	View em Android	View em iOS	Análogo na Web	Descrição
<View>	<ViewGroup>	<UIView>	Um sem rolagem (non-scrolling)	Um container que suporta `layout` com `flexbox`, `style`, algum `touch handling`, e control de acessibilidade
<Text>	<TextView>	<UITextView>	<p>	Exibe `styles` e `strings` aninhados de texto e `handles touch events`
<Image>	<ImageView>	<UIImageView>	<img>	Exibe diferentes tipos de imagens
<ScrollView>	<ScrollView>	<UIScrollView>		Container genérico de `scrolling` que contém multiplos `components` e `views`.
<TextInput>	<EditText>	<UITextField>	<input type="text">	Permite que o usuário insira texto

Encontre a lista completa de componentes core de RN em: https://reactnative.dev/docs/components-and-apis

Processo de Desenvolvimento de uma Aplicação Web/Mobile

https://azure.microsoft.com/pt-br/resources/cloud-computing-dictionary/what-is-mobile-app-development



Identificação da idéia

Pesquisa de mercado e análise de viabilidade de negócio

Identificação dos Requisitos Funcionais

Entender a jornada do usuário

Projetar as interfaces de usuário

Criar um protótipo

Validar o *design* com o usuário

Escolher o *stack* tecnológico

Escolher a arquitetura do App

Projetar o banco de dados e os componentes lógicos

Codificar o *front-end*

Codificar o *back-end*

Codificar *APIs*

Integrar *front* e APIs *back-end*

Documentar

Definir os casos de teste

Codificar os casos de teste

Executar os casos de teste

Corrigir erros

Definir a infraestrutura física (servidores locais, na nuvem, etc.)

Gerar/Compilar os executáveis ou gerar o projeto

Instalar na infraestrutura ou publicar nas lojas de Apps

Comparação de tipos de Apps móveis

	Aplicativos nativos	Aplicativos multiplataforma	Aplicativos Web progressivos	Aplicativos híbridos
Número de bases de código	Um por plataforma	Um, mas compilado para cada plataforma	Só um	Um para o aplicativo, outro para o contêiner
Linguagens e estruturas	Apenas nativo	Escolha da equipe	Somente Web	Web e nativo
Acesso a SDKs e APIs	Sim	Sim	Não	Limitado
Desempenho	Maior	Alto	Menor	Baixo
Acesso ao hardware do dispositivo	Completo	Maioria	Muito pouco	Alguns
Capacidade de resposta à entrada do usuário	Bom	Bom	Pior	Ruim
Interatividade	Alto	Alto	Menor	Baixo
Uso de recursos do dispositivo	Alto	Alto	Baixo	Médio
Requer conectividade	Não (Sim, depende)	Não (Sim, depende)	Sim	Sim
Custo para criar e manter	Maior	Alto	Menor	Inferior
Onde o aplicativo é armazenado	Dispositivo	Dispositivo	Servidor	Dispositivo e servidor
Implantado por meio de	Marketplace	Marketplace	Navegador	Marketplace
Requer aprovação externa	Sim	Sim	Não	Sim

RELATED

Weighing the Trade-Offs of Native vs. Cross-Platform Mobile App Development

Boosting Efficiency With Cross-Platform Code-Sharing Techniques

Unleashing Cross-Platform App Development: Mastering the Vital Tech Stack for Unprecedented Success

Emerging Cross-Platform Tools for Building Better, More Integrated Mobile Apps

Native vs Hybrid vs Cross-Platform: How and What to Choose?

What are the similarities and differences between native, hybrid, and cross-platform development and how do you know which to choose?

By Sanjay Kidecha · Updated Jul. 27, 22 · Analysis

Like (8) Comment (3) Save Tweet Share 36.7K Views

Join the DZone community and get the full member experience. JOIN FOR FREE

After coming up with an online business idea, many people get confused between the app development approaches. It is crucial for them to decide on the right approach from the Native, Hybrid, and Cross-platform mobile apps. The reason that each type of app has its specification, pros, and cons, this confusion is reasonable. Finding out what type of app building approach will connect the audience with the brand faster is a real hassle.

You must know choosing the right harvesting ground is essential for successful outcomes. Hence, it is no brainer that choosing the right platform for mobile app development is paramount, and it depends entirely on the nature of your business.

Nevertheless, many businesses end up making a mistake when choosing the mobile app development approach. It happens even after they were conscious of their business nature. Choosing the wrong development approach causes unproductive software products for your online business.

Therefore, we have curated the difference between Native Vs. Hybrid Vs. Cross-platform to clear this trilemma in the blog. Don't miss any line!

TRENDING

The Beginner's Guide To Understanding Graph Databases

Microservices Design Patterns for Highly Resilient Architecture

Snowflake Data Time Travel

When It's Time to Give REST a Rest

https://dzone.com/articles/native-vs-hybrid-vs-cross-platform-how-and-what-to

Design leve

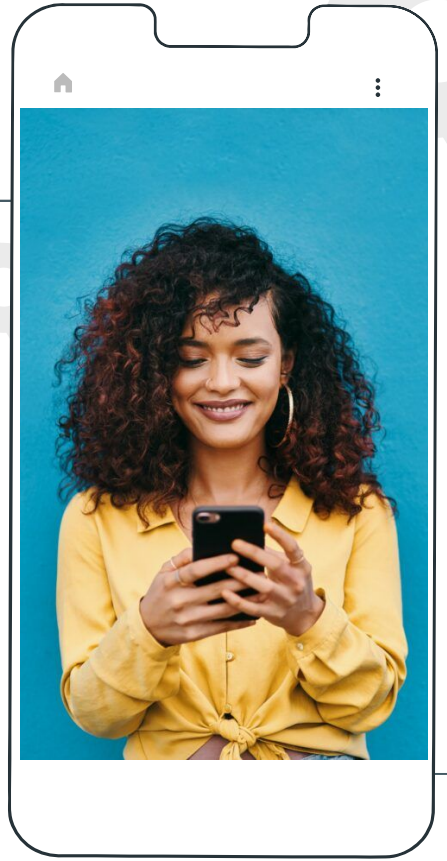
Não importa se escolher o desenvolvimento de aplicativos nativos ou híbridos, um dos primeiros obstáculos que precisam ser superados são os recursos relativamente limitados dos dispositivos móveis.

Ao longo do processo de desenvolvimento de aplicativos, é preciso assegurar que os Apps consumam menos recursos que um aplicativo desktop típico.

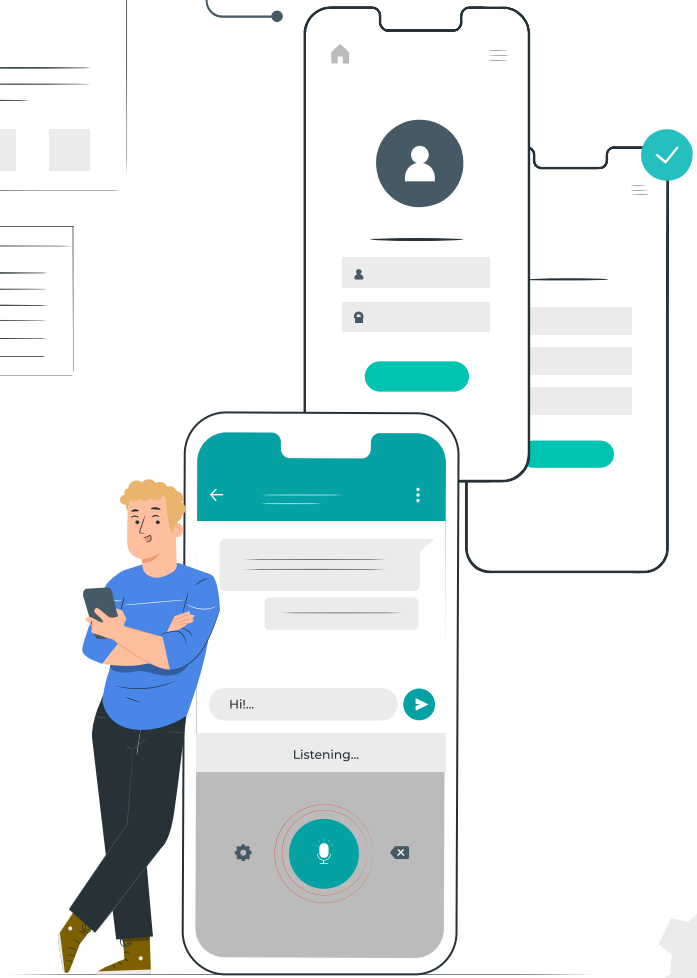
É essencial oferecer uma ótima experiência ao usuário. A interface com o usuário de um aplicativo móvel deve ser mais simples do que uma interface de aplicativos desktop.

A interface deve ser projetada para toque. Os usuários móveis devem ser capazes de navegar seu aplicativo facilmente e inserir dados sem digitação excessiva.

Usuários buscam aplicativos que, acima de tudo, sejam convenientes e fáceis de usar em seus dispositivos móveis.

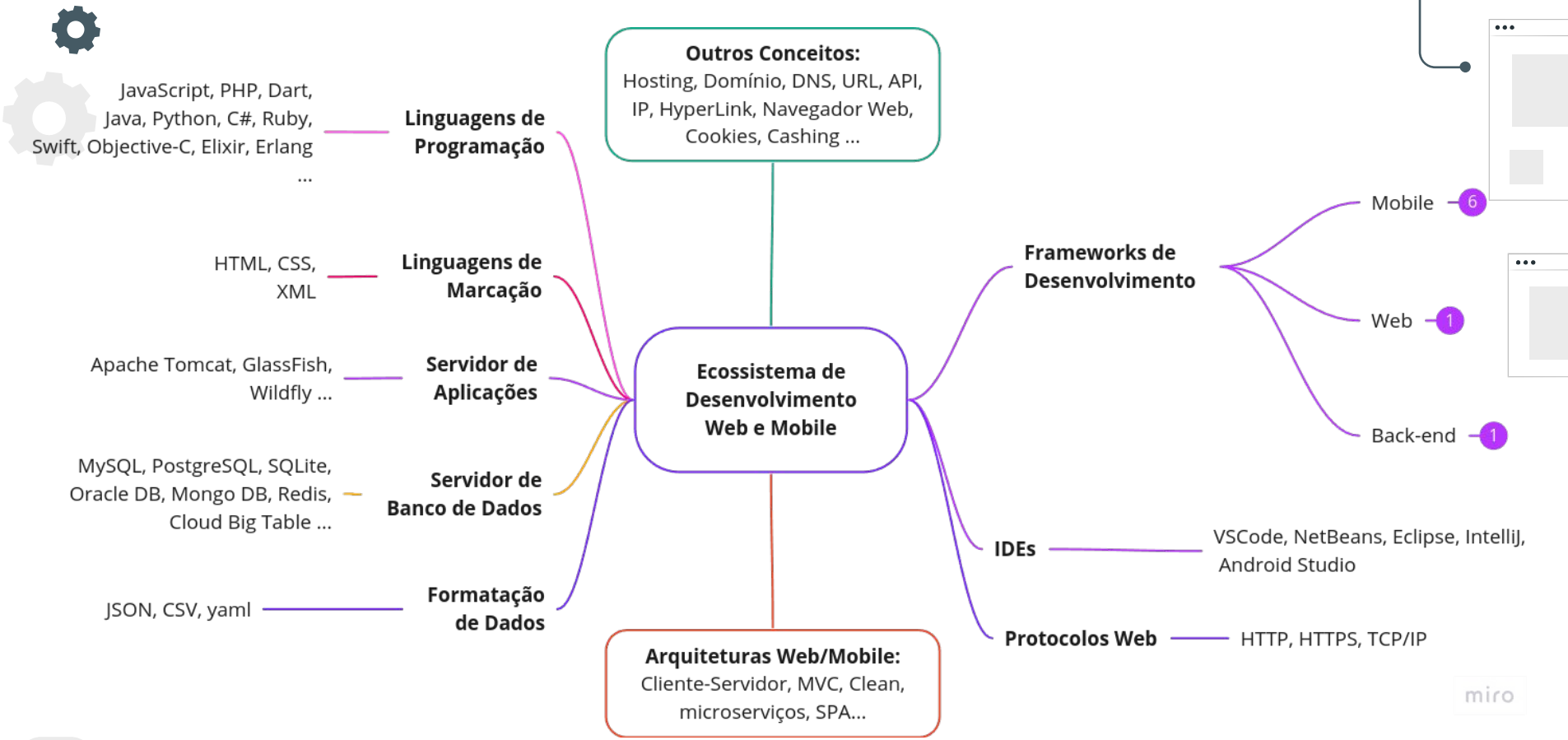


https://www.ibm.com/br-pt/topics/mobile-application-development



02

Ecossistema de desenvolvimento mobile


Outros Conceitos:
Hosting, Domínio, DNS, URL, API, IP, HyperLink, Navegador Web, Cookies, Cashing ...
Ecossistema de Desenvolvimento Web e Mobile
Linguagens de Programação
JavaScript, PHP, Dart, Java, Python, C#, Ruby, Swift, Objective-C, Elixir, Erlang ...
Linguagens de Marcação
HTML, CSS, XML
Servidor de Aplicações
Apache Tomcat, GlassFish, Wildfly ...
Servidor de Banco de Dados
MySQL, PostgreSQL, SQLite, Oracle DB, Mongo DB, Redis, Cloud Big Table ...
Formatação de Dados
JSON, CSV, yaml
Frameworks de Desenvolvimento
Mobile
6
Web
1
Back-end
1
IDEs
VSCode, NetBeans, Eclipse, IntelliJ, Android Studio
Protocolos Web
HTTP, HTTPS, TCP/IP
Arquiteturas Web/Mobile:
Cliente-Servidor, MVC, Clean, microserviços, SPA...
miro

Frameworks de Desenvolvimento

- Mobile
  - Multiplataforma* — Flutter, React Native, Kotlin Multiplatform Ionic, NativeScript, Xamarin, Cordova
  - iOS — SwiftUI, UIKit
  - Android — Android Studio
- Web — React, Angular, Vue.js, Svelte, JQuery, Semantic UI, Ember, Foundation, Preact, ...
- Back-end — ASP.NET Core, Django, Laravel, Rails, Express.js, Flask, Spring Boot, Phoenix ...

miro

* Alguns frameworks também são usados para desenvolvimento de Apps híbridos e/ou web progressivos

03

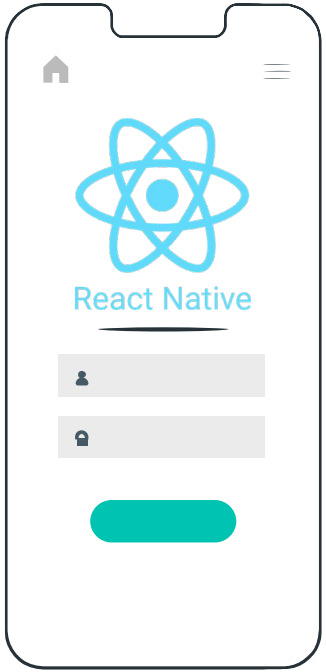
Introdução ao React Native - RN

React Native - RN

React Native é um framework de software de UI (front-end) de código aberto criada pela Meta Platforms, Inc em 2015.

É usado para desenvolver aplicativos para Android, Android TV, iOS, macOS, tvOS, Web, Windows e UWP*, permitindo que os(as) desenvolvedores(as) usem o framework React junto com os recursos nativos da plataforma.

RN Ele é usado para desenvolver aplicativos Android e iOS no Facebook, Microsoft e Shopify. Também está sendo usado para desenvolver aplicativos de realidade virtual na Oculus.



* UWP - Unified Windows Platform

O conceito de View

No desenvolvimento Android e iOS, uma **view** é o bloco de construção básico da UI: um pequeno elemento retangular na tela que pode ser usado para exibir texto, imagens ou responder à entrada do usuário.

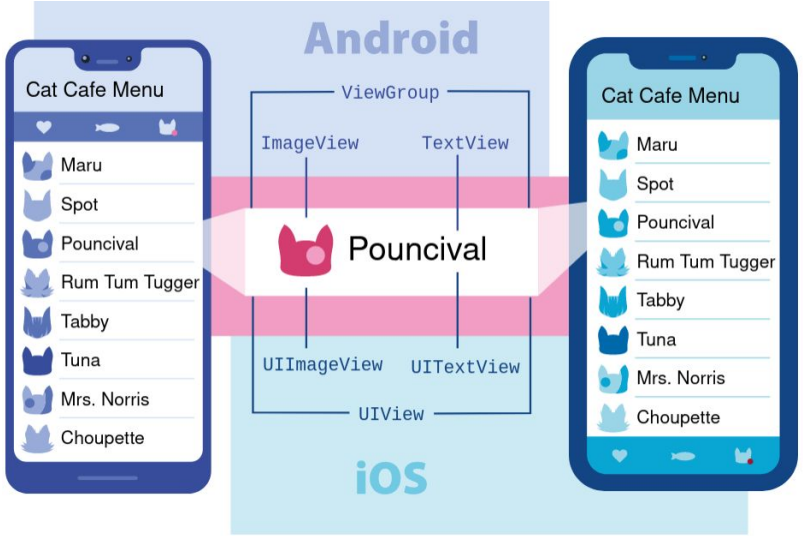
Mesmo os menores elementos visuais de um aplicativo, como uma linha de texto ou um botão, são tipos de **views**.

Alguns tipos de **views** podem conter outras **views**. São **views** até o fim!

Para desenvolvimento em Android as **views** são programadas em Kotlin ou Java.

Em iOS é usado Swift ou Objective-C.

Em RN as **views** são programadas com JavaScript usando **componentes** do framework React.



Just a sampling of the many views used in Android and iOS apps.

https://reactnative.dev/docs/intro-react-native-components

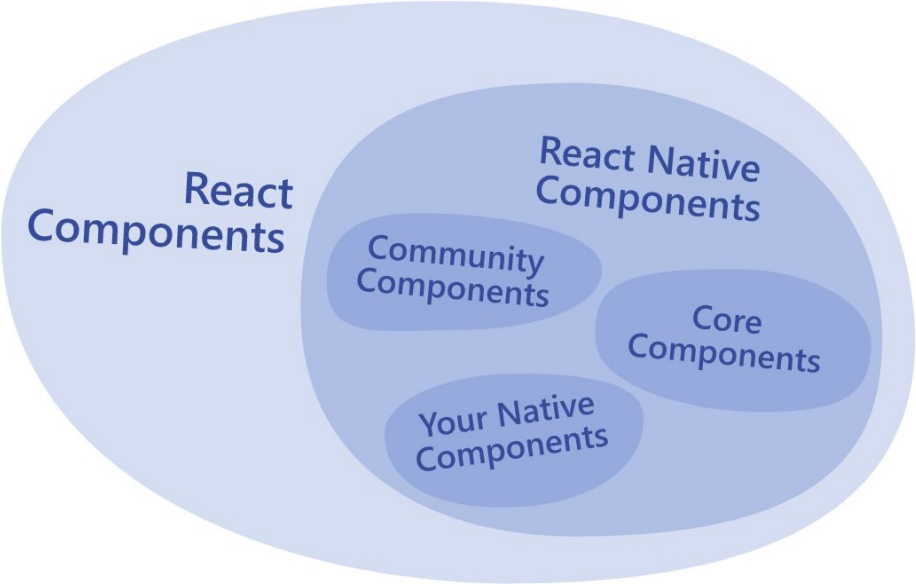
Componentes Core em RN

Componente de UI do RN	View em Android	View em iOS	Análogo na Web	Descrição
<View>	<ViewGroup>	<UIView>	Um sem rolagem (non-scrolling)	Um container que suporta `layout` com `flexbox`, `style`, algum `touch handling`, e control de acessibilidade
<Text>	<TextView>	<UITextView>	<p>	Exibe `styles` e `strings` aninhados de texto e `handles touch events`
<Image>	<ImageView>	<UIImageView>	<img>	Exibe diferentes tipos de imagens
<ScrollView>	<ScrollView>	<UIScrollView>		Container genérico de `scrolling` que contém multiplos `components` e `views`.
<TextInput>	<EditText>	<UITextField>	<input type="text">	Permite que o usuário insira texto

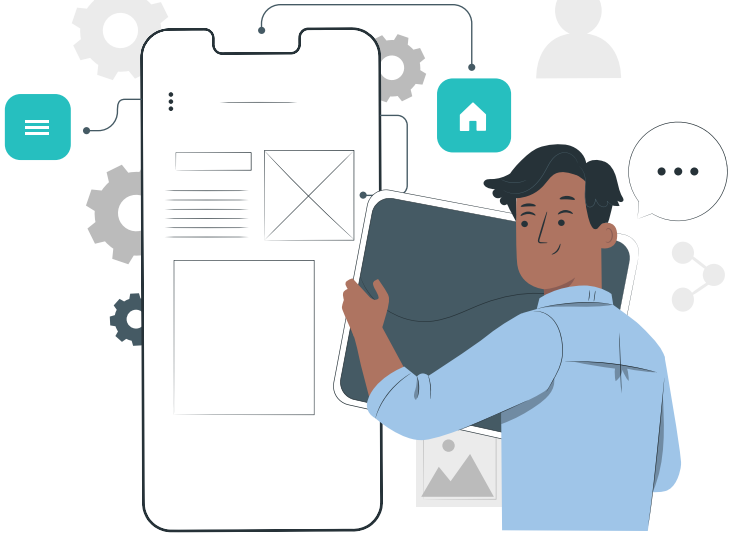
Encontre a lista completa de componentes core de RN em: https://reactnative.dev/docs/components-and-apis

Componentes Core em RN

React Native utiliza a mesma estrutura de APIs dos componentes do framework React.



Encontre a lista completa de componentes core de RN em: https://reactnative.dev/docs/components-and-apis



04

Arquitetura do React Native - RN

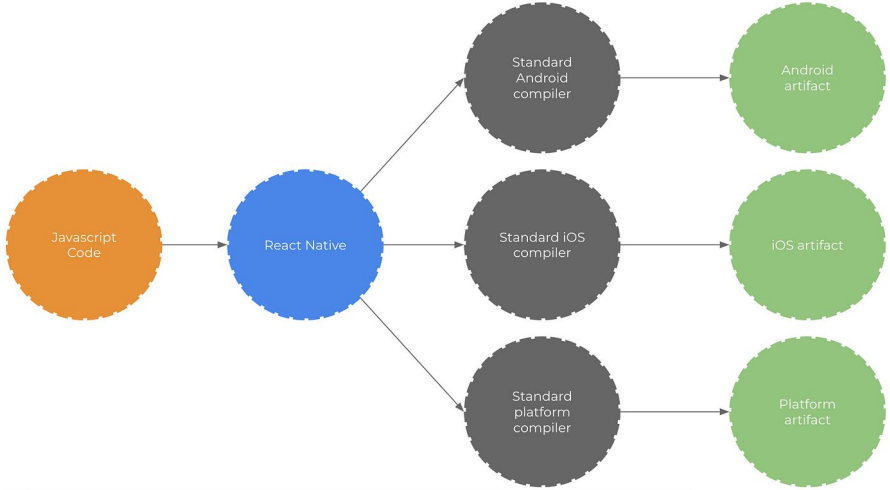
Arquitetura React Native

Hoje RN possui duas principais arquiteturas:

- **Bridge** (Primeira versão da arquitetura do React Native, desde 2018)

- **Fabric** (Nova arquitetura beta, disponível a partir da versão 0.68, com melhorias de desempenho e comunicação do framework, lançada em Abril de 2024)

Objetivo: Permitir a comunicação do código JS em RN com o código nativo do aparelho (Android/iOS)



Arquitetura React Native - BRIDGE

A bridge é o coração do React-Native ela faz uma comunicação bidirecional e assíncrona com o dispositivo, a comunicação funciona da seguinte forma :

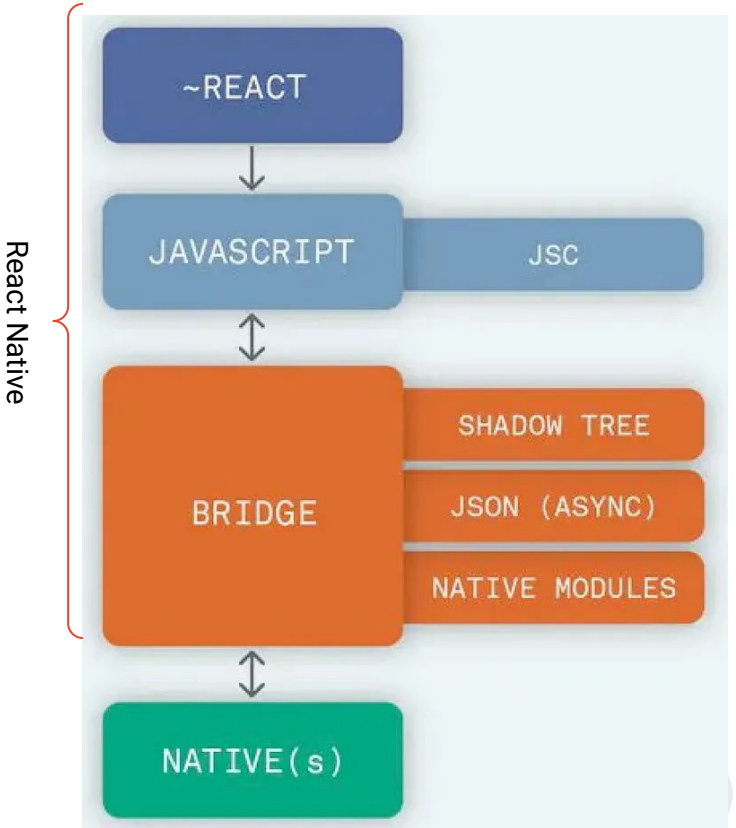
1: Escrevemos o código JavaScript no nosso editor de texto por exemplo um <Button />

2: O JavaScriptCore (JSC) compila esse código e envia para a bridge

3: A bridge envia essas informações para o código nativo solicitando que renderize esse <Button />

4: O nativo entende e renderiza o componente em tela

****5: Caso haja um clique nesse botão quem recebe esse evento primeiramente é o nativo depois essa informação é passada para a bridge novamente para que o React-Native faça as ações programadas no JavaScript***



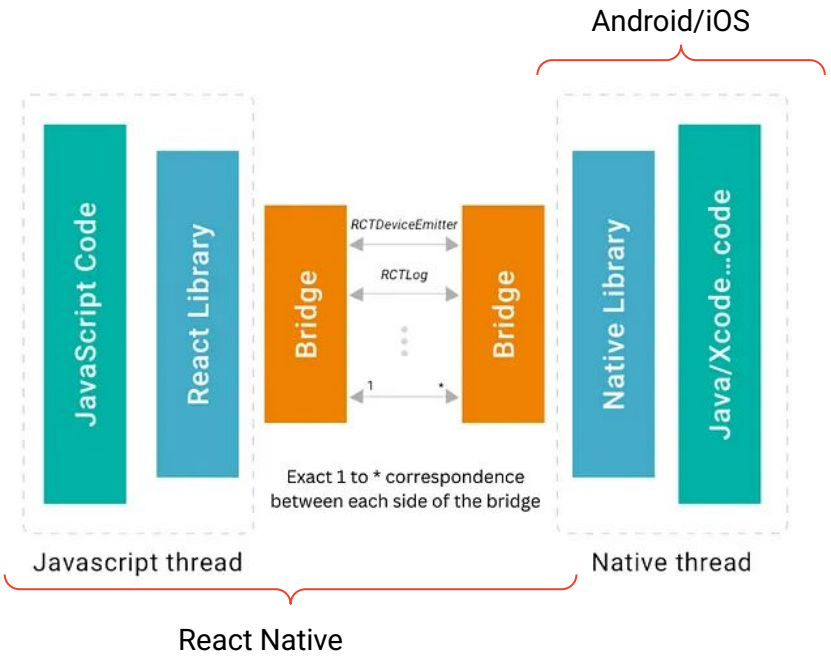
Descrição detalhada da arquitetura bridge: https://medium.com/syngentadigital-insights/react-native-do-bridge-ao-fabric-como-essa-arquitetura-funciona-por-dentro-parte-01-391e87ee615c

Arquitetura React Native - BRIDGE

Problemas no Bridge que motivaram a criação do Fabric:

- Ter um middleware entre a tomada de decisão e a decisão em si possui impactos como por exemplo uma sobrecarga nas filas de mensagens (json) a serem processadas, que podem causar side-effects na aplicação.

- Outros problemas como **carregarmos os Native Modules ainda no load inicial da aplicação e não apenas quando formos usar,** também nos traz problemas de performance no load inicial do App.



Descrição detalhada da arquitetura bridge: https://medium.com/syngentadigital-insights/react-native-do-bridge-ao-fabric-como-essa-arquitetura-funciona-por-dentro-parte-01-391e87ee615c

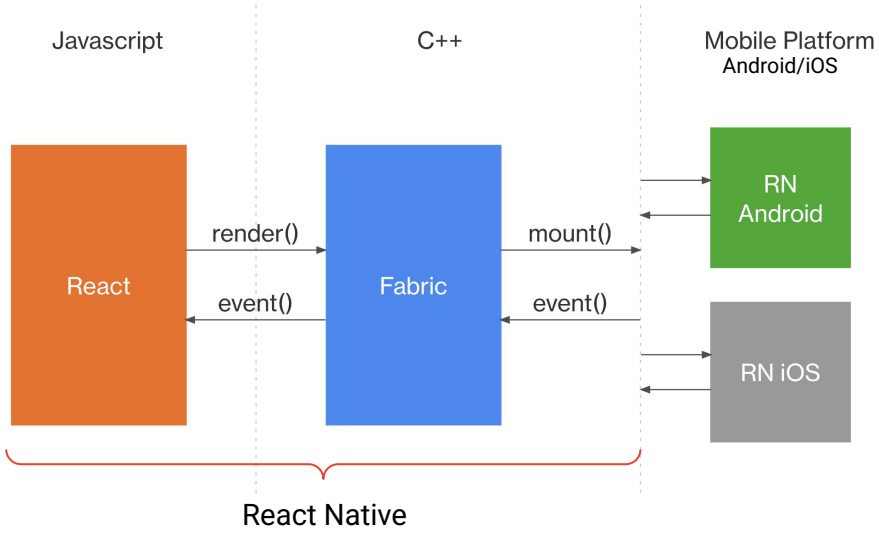
Nova Arquitetura React Native - FABRIC

Fabric é o novo sistema de renderização do React Native, uma evolução conceitual do sistema de renderização legado.

Os princípios básicos são **unificar mais lógica de renderização em C++, melhorar a interoperabilidade com plataformas *host* (Android/iOS) e desbloquear novos recursos para React Native.**

Melhorias:

- **Segurança na tipagem de dados** durante a conversão (JS -> C++ -> Java/Kotlin/Swift...)
- **Core C++ compartilhado** entre o render e as plataformas *host*
- Melhor **interoperabilidade** com as plataformas *host*
- Melhor **desempenho** (view flattening)
- Facilidade para **manter consistência** dos códigos entre as diversas plataformas
- **Startup rápido** (ao contrário do bridge)



Documentação da nova arquitetura: https://reactnative.dev/architecture/overview

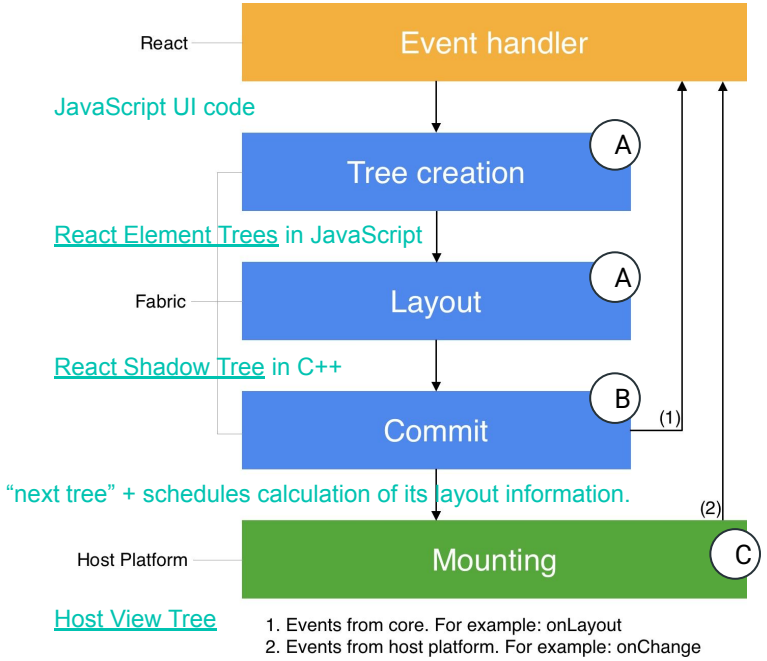
Nova Arquitetura React Native - FABRIC

Render pipeline:

A. ***Render:*** RN executa a lógica do produto que cria React Element Trees em JavaScript. A partir desta árvore, o renderizador cria uma React Shadow Tree em C++.

B. ***Commit:*** Depois que um React Shadow Tree é totalmente criado, o renderizador aciona um commit. Isso promove tanto o React Element Tree quanto o recém-criado React Shadow Tree como a "next tree" a ser montada. Isso também agenda o cálculo de suas informações de layout (size, color...)

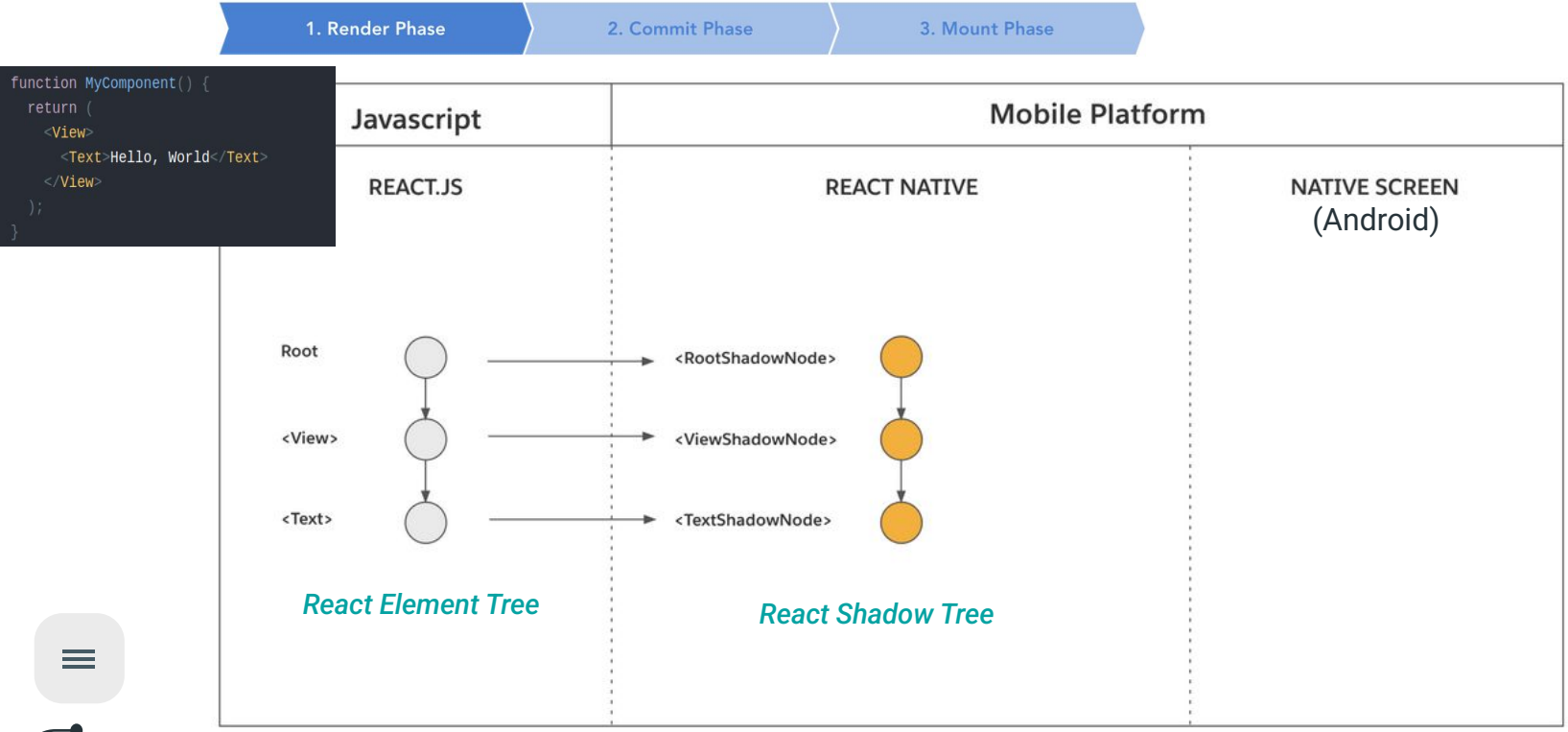
C. ***Mount:*** A React Shadow Tree, agora com os resultados do cálculo do layout, é transformada em uma Host View Tree.



Documentação da nova arquitetura: https://reactnative.dev/architecture/overview

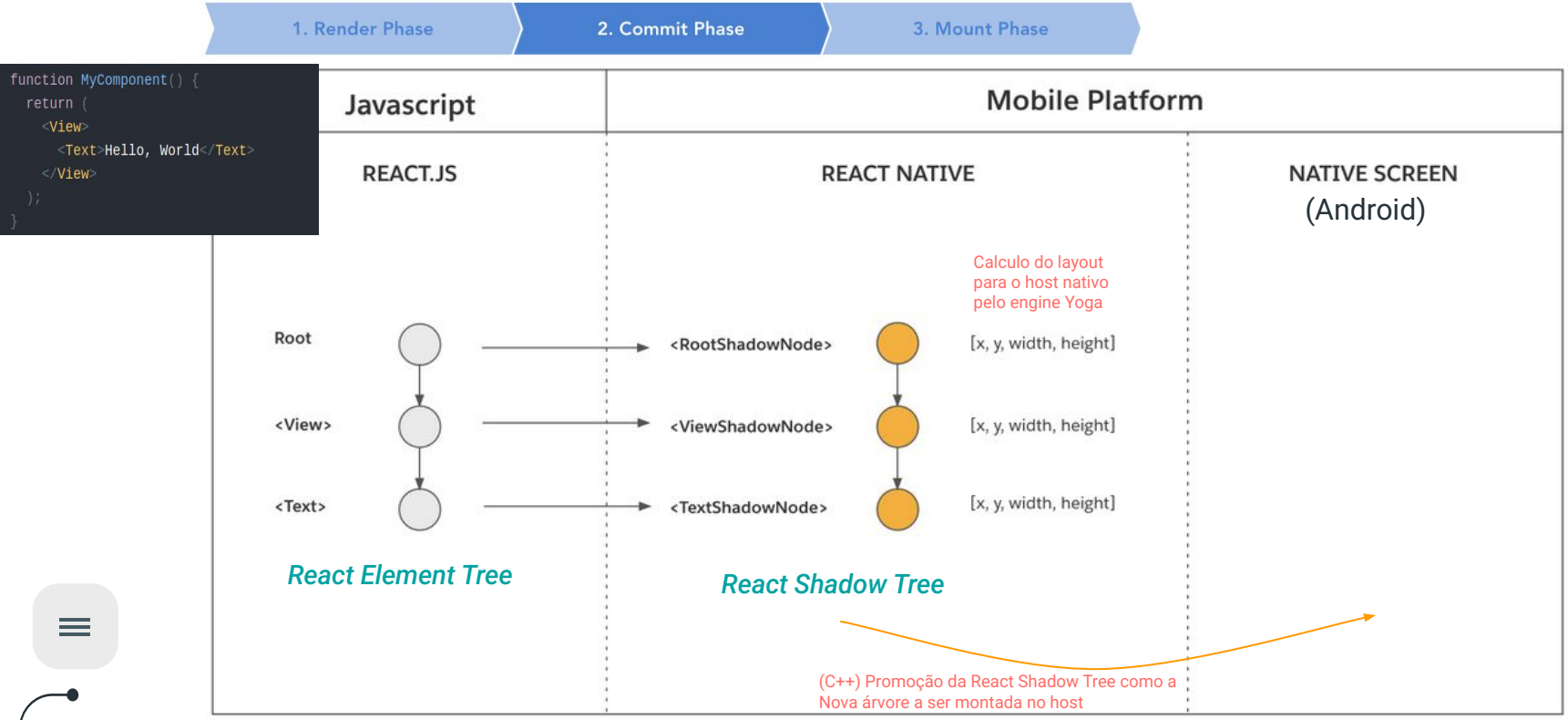
Nova Arquitetura React Native - FABRIC

Estado Inicial da React Element Tree - Etapa 1 - Renderização



Nova Arquitetura React Native - FABRIC

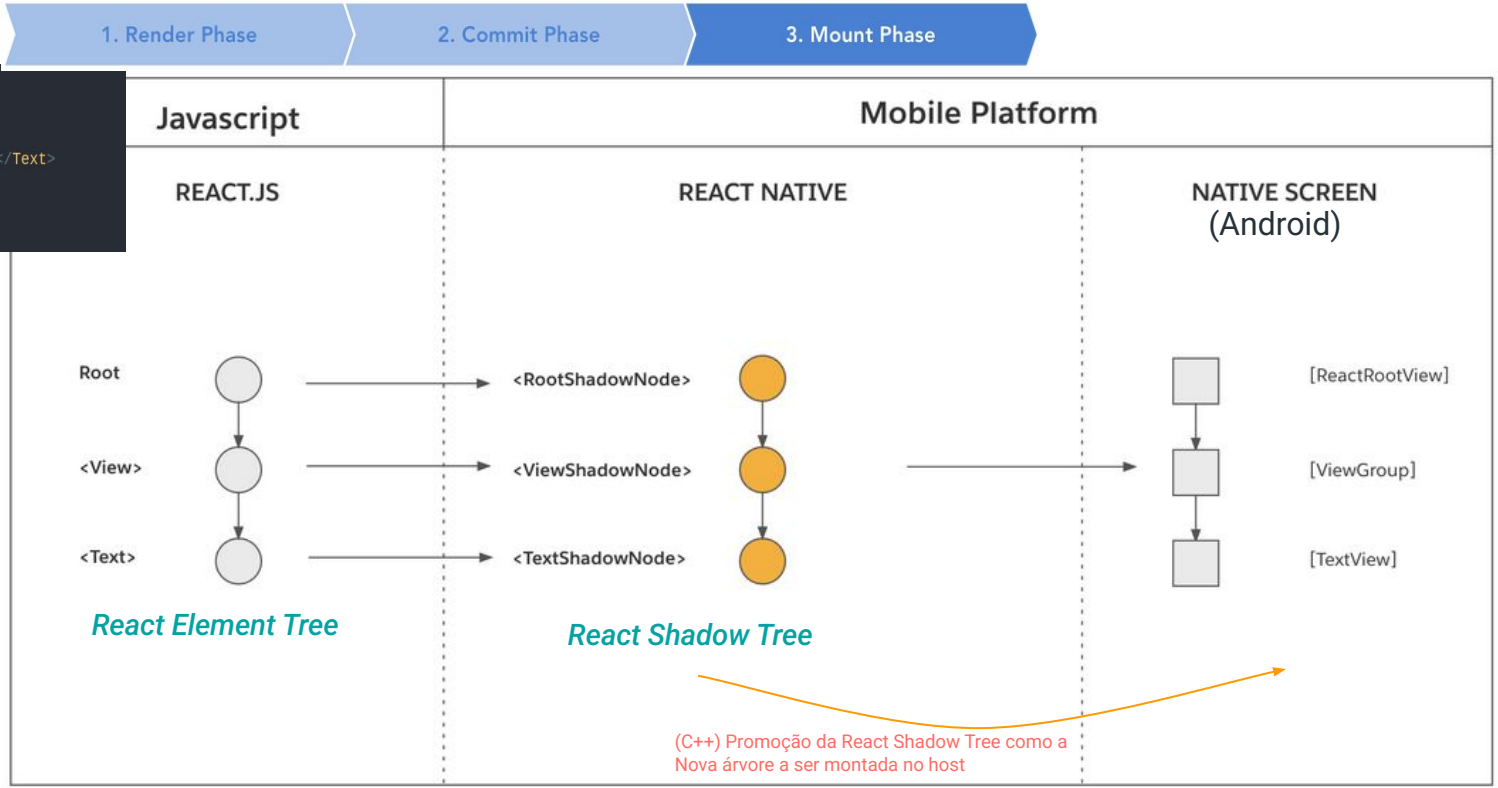
Estado Inicial da React Element Tree - Etapa 2 - Commit



Nova Arquitetura React Native - FABRIC

Estado Inicial da React Element Tree - Etapa 3 - Montagem

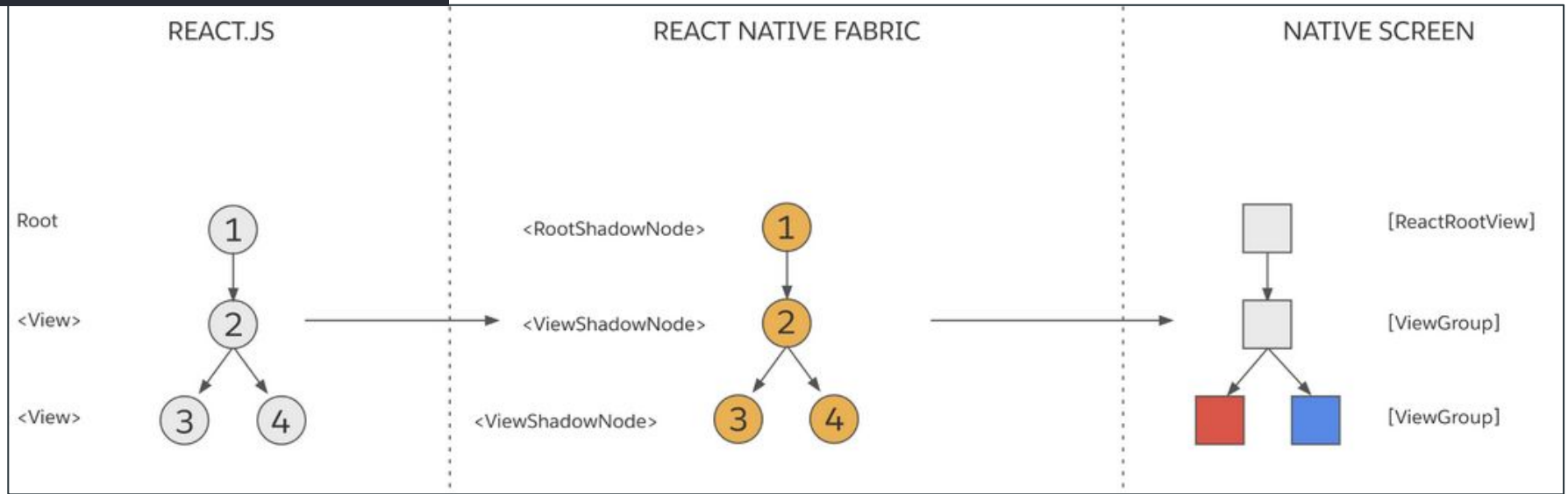
```
function MyComponent() {
  return (
    <View>
      <Text>Hello, World</Text>
    </View>
  );
}
```



Nova Arquitetura React Native - FABRIC

Atualização de Estado da React Element Tree

```
function MyComponent() {
  return (
    <View>
      <View
        style={{backgroundColor: 'red', height: 20, width: 20}}
      />
      <View
        style={{backgroundColor: 'blue', height: 20, width: 20}}
      />
    </View>
  );
}
```



Nova Arquitetura React Native - FABRIC

Atualização de Estado da React Element Tree

```
function MyComponent() {
  return (
    <View>
      <View
        style={{backgroundColor: 'red', height: 20, width: 20}}
      />
      <View
        style={{backgroundColor: 'blue', height: 20, width: 20}}
      />
    </View>
  );
}
```

Estado novo →

```
<View>
  <View
    style={{backgroundColor: 'yellow', height: 20, width: 20}}
  />
  <View
    style={{backgroundColor: 'blue', height: 20, width: 20}}
  />
</View>
```

Quando React cria uma nova *React Element Tree* que incorpora o novo estado deve **clonar** cada elemento React (View, Text, ..) e nó na React Shadow Tree que será impactado pela mudança.

Após clonar, a nova React Shadow Tree é *commited* e montada.

React cria a nova **React Element Tree (T')** usando as seguintes operações:

1. CloneNode(**Node 3**, `{backgroundColor: 'yellow'}`) → **Node 3'**
2. CloneNode(**Node 2**) → **Node 2'**
3. AppendChild(**Node 2'**, **Node 3'**)
4. AppendChild(**Node 2'**, **Node 4**)
5. CloneNode(**Node 1**) → **Node 1'**
6. AppendChild(**Node 1'**, **Node 2'**)

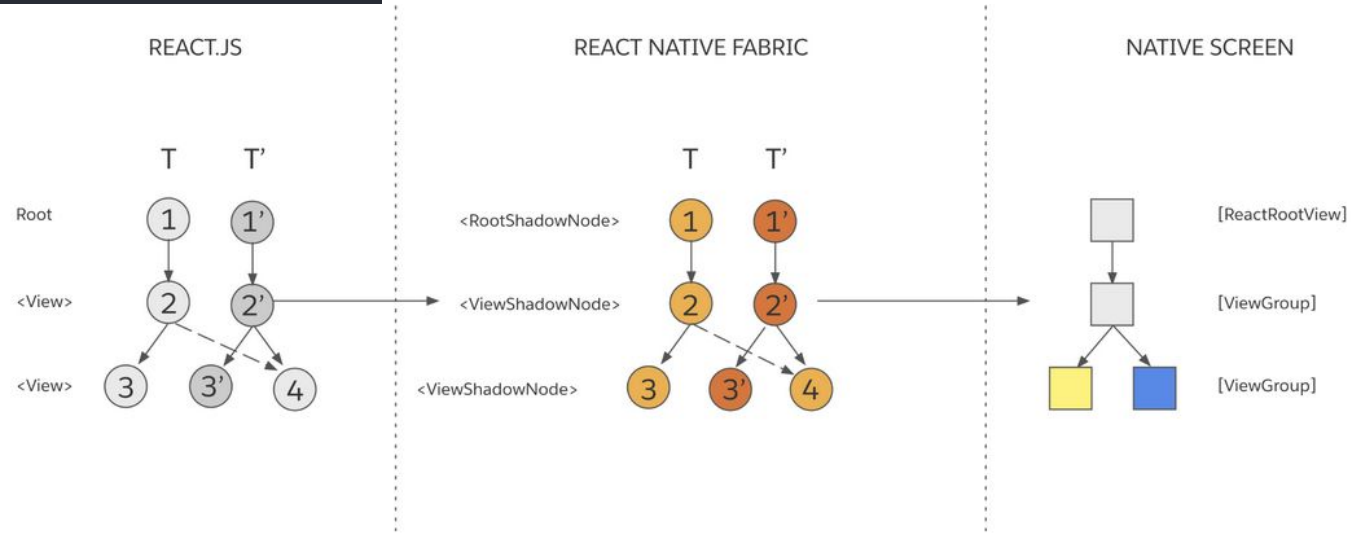
Nova Arquitetura React Native - FABRIC

Atualização de Estado da React Element Tree

```
function MyComponent() {
  return (
    <View>
      <View
        style={{backgroundColor: 'red', height: 20, width: 20}}
      />
      <View
        style={{backgroundColor: 'blue', height: 20, width: 20}}
      />
    </View>
  );
}
```

Estado novo

```
<View>
  <View
    style={{backgroundColor: 'yellow', height: 20, width: 20}}
  />
  <View
    style={{backgroundColor: 'blue', height: 20, width: 20}}
  />
</View>
```



04

Projeto em React Native - RN

Projeto
Atividade da Semana

- Escolher uma ideia de App mobile com a equipe. Pensem em soluções a problemas do dia a dia de vocês (p.ex. Organização de tempo, gestão financeira, gestão de república, organização de atividades, gerenciamento de notas e frequência nas disciplinas, saúde física, saúde mental, etc.)

- Fazer protótipos das UI em figma (https://www.figma.com) que serão desenvolvidas nas próximas 5 semanas, sendo que, pelo menos duas interfaces deverão realizar manipulação de dados de banco de dados (inserção, consulta, modificação, etc..)
    - **Dica:** planejar duas UI por integrante da equipe.

Tutorial de Figma para design UI/UX:
https://www.youtube.com/watch?v=BOt3MNB71gI&list=PLjiHFwhbHYlEmPhn68XdG2p2k4X47XR-8

Entregar um documento que apresente a ideia e as IU, descrevendo as funcionalidades pensadas para cada UI. Entregar no e-disciplinas a modelagem das UI em figma até dia 27 de Maio.

Referências

O que é desenvolvimento de aplicativos móveis?

- https://www.ibm.com/br-pt/topics/mobile-application-development
- https://azure.microsoft.com/pt-br/resources/cloud-computing-dictionary/what-is-mobile-app-development

Introdução ao React Native: https://reactnative.dev/docs/getting-started

Arquitetura do React Native: https://reactnative.dev/architecture/overview

React Native: do Bridge ao Fabric, como essa arquitetura funciona? — Parte 01
https://medium.com/syngentadigital-insights/react-native-do-bridge-ao-fabric-como-essa-arquitetura-funciona-por-dentro-parte-01-391e87ee615c

Native vs Hybrid vs Cross-Platform: How and What to Choose?:
https://dzone.com/articles/native-vs-hybrid-vs-cross-platform-how-and-what-to